



Scaling Django: Mastering Database Routers with AWS Aurora

Leonardo Batista - August 2026

About me



- Solutions Architect, DevSecOps Lead and many other functions at OneBlink
- AWS Certified Developer Associate and Solutions Architect Associate
- 5 years working (a.k.a. in love) with Django
- 3rd DjangoCon US (1st in-person)
1st conference as speaker



The breaking point

First, the context



OneBlinc

OneBlinc is a US-based fintech startup focused on subprime / underserved people, usually ignored or exploited by big banks.

The company started by providing payroll loans for federal employees, but our current main product is **BlincAdvance**, a cash advance app.



Small, high-performance and multi-purpose team.



Lean infrastructure, focusing on managed services and automation.



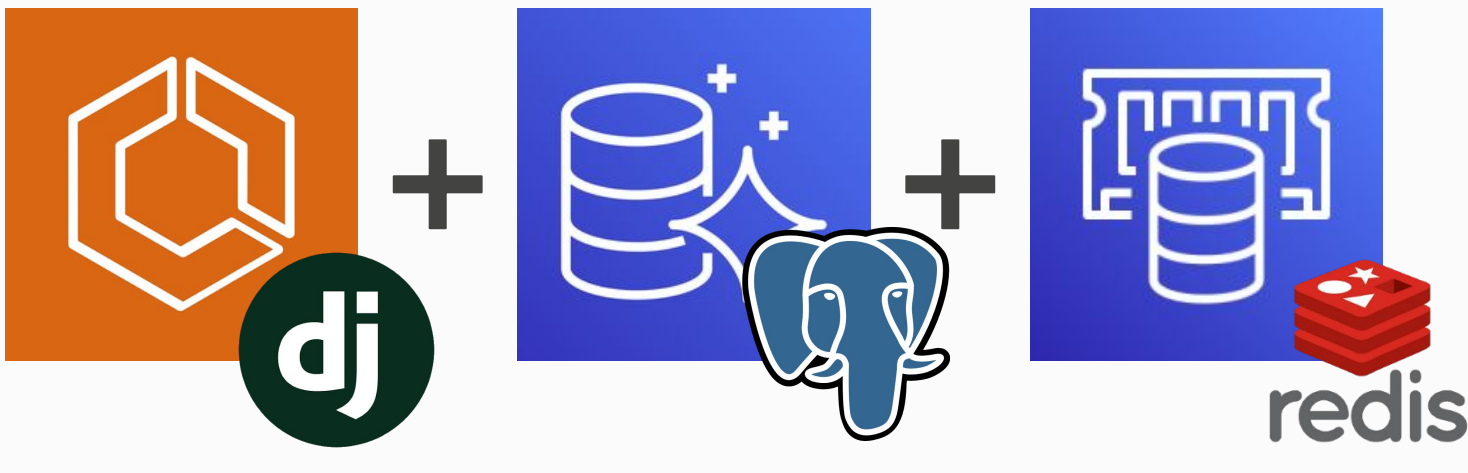
Growing fast, scalability becoming a real problem



First, the context

The core backend stack:

- Django + DRF for both API and Internal Tools, running on Amazon ECS (w/ auto-scaling)
- Aurora PostgreSQL for the DB
- Redis for caching



The problem

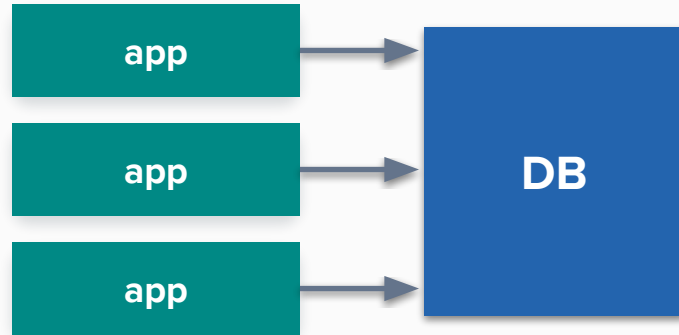
- Backend becoming overloaded at peak hours and during heavy batches → Increase number of nodes
- Backend stabilized → Database becomes overloaded
- **What to do?**
 - Increase DB size?
 - Add indexes?



The DB is your first bottleneck

You scale the app horizontally in an afternoon.
The database doesn't follow for free.

Every container points at one primary. Reads and writes pile onto a single node — and that node is your ceiling.



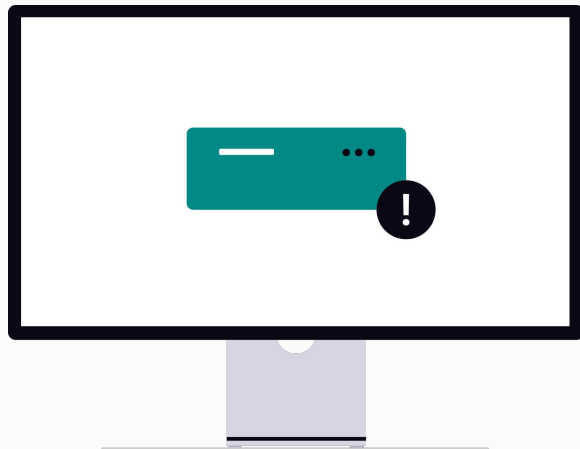
N containers, one ceiling



Symptoms

Your app scales. Your database doesn't

- App latency climbs (p95/p99) while throughput stays flat
- Workers stuck waiting on the DB, not on CPU
- Connection pool saturation - "too many connections"
- Retry storms → cascading 5xx
- Adding app instances does nothing (or makes it worse)



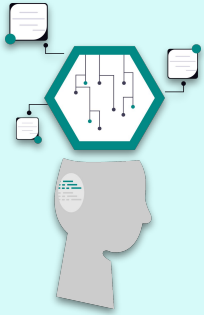
Symptoms

One instance, doing everything

- CPU pinned, no headroom for spikes
- Active sessions > cores → queries queue for CPU
- Reads queuing behind writes
- Cache hit ratio falling, disk IOPS rising
- Lock contention, deadlocks, vacuum falling behind



Symptoms



AI Tip:

- Correlate connection error logs with backend and database metrics to identify critical paths, usage peaks and slow queries.



Why Aurora?

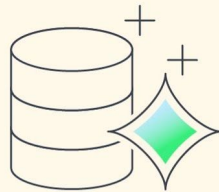
Before why, what's Aurora?

Amazon Aurora is a managed relational database engine from AWS, available on RDS.

- MySQL / PostgreSQL compatible - your Django app doesn't know the difference
- Compute and storage are decoupled
- Shared storage layer, replicated 6 ways across 3 AZs
- Storage auto-grows in 10 GB steps (up to 128 TiB)
- Up to 15 read replicas, all reading the same storage



[AWS Aurora Docs](#)



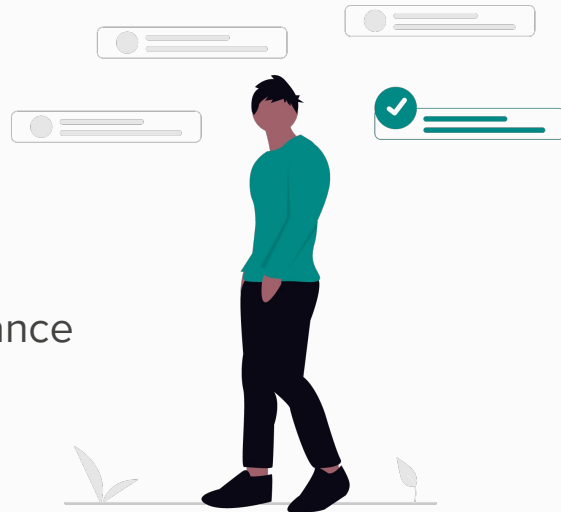
Amazon
Aurora



Why did we choose it?

- Replicas share storage → very low lag (typically ~10-20 ms)
- **Writer fails → a reader gets promoted fast**
 - **Also makes maintenance and scaling easier**
- Two stable endpoints: cluster (writer) + reader
- **Automatic load-balancing across reader instances**

Trade-off: Cost → More expensive than an equivalent RDS instance



The router

What is a DB router?

- A class that picks which connection an operation uses
- Unlocks multiple DATABASES aliases instead of one default
- Registered in DATABASE_ROUTERS - consulted in order
- Return an alias, or None to defer to the next router
- Only selects a connection → doesn't manage transactions or override `.using()`



[Django Docs:
Multiple Databases](#)



The 4 methods

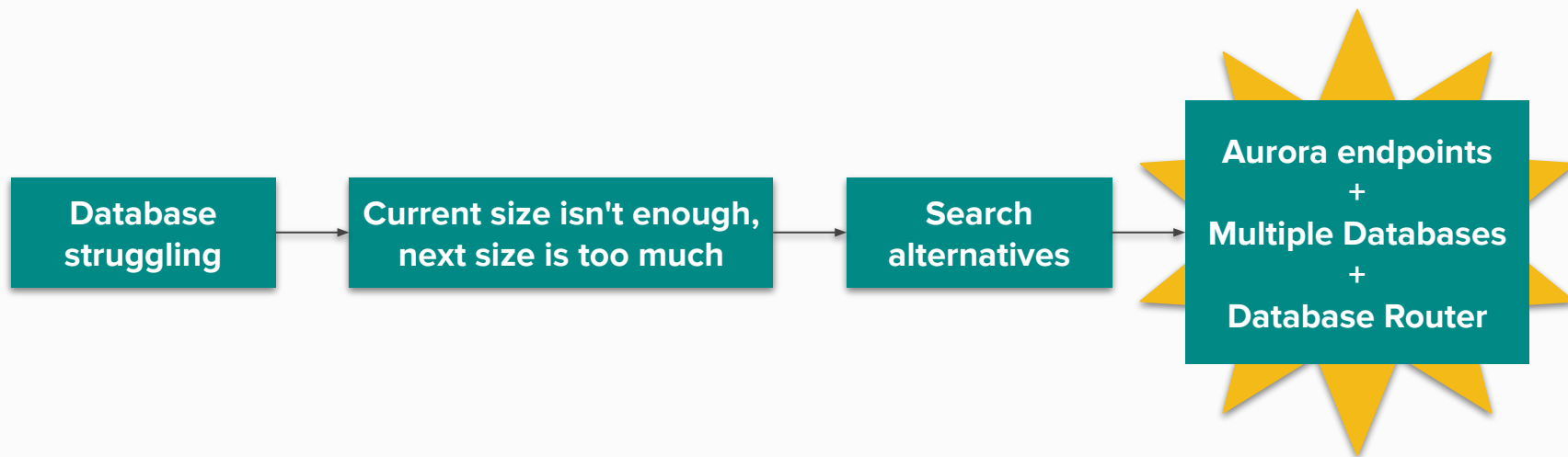
- **db_for_read** - which alias serves a read
- **db_for_write** - which alias takes a write
- **allow_relation** - is a relation between two objects allowed
- **allow_migrate** - should this model migrate on this alias



```
1 class MyRouter:
2     def db_for_read(self, model, **hints): ...
3     def db_for_write(self, model, **hints): ...
4     def allow_relation(self, obj1, obj2, **hints): ...
5     def allow_migrate(self, db, app_label,
6                       model_name=None, **hints): ...
7
```



The idea that changed everything



Setting up the connections

First, we need to configure the 2 DB connections in our **DATABASES** setting:

- **default** points to the writer (cluster endpoint)
- **readonly** will point (obviously) to read-replicas endpoint

```
1  DATABASES = {
2      "default": {
3          "ENGINE": "django.db.backends.postgresql",
4          "HOST": "my-cluster.cluster-xxxxxx.us-east-1.rds.amazonaws.com",
5          "PORT": "5432",
6          "NAME": "dbname",
7          "USER": "username",
8          "PASSWORD": "password",
9      },
10     "readonly": {
11         "ENGINE": "django.db.backends.postgresql",
12         "HOST": "my-cluster.cluster-ro-xxxxxx.us-east-1.rds.amazonaws.com",
13         "PORT": "5432",
14         "NAME": "dbname",
15         "USER": "username",
16         "PASSWORD": "password",
17     },
18 }
```



Building the custom router

First, let's create the first two methods: the "selectors".

```
1 class ReadReplicaRouter:
2     def db_for_read(self, model, **hints):
3         """
4         Reads go to the Aurora-managed read replicas.
5         """
6         return "readonly"
7
8     def db_for_write(self, model, **hints):
9         """
10        Writes always go to the primary instance (writer).
11        """
12        return "default"
```



Building the custom router

Then, we add the "validators":

```
1 class ReadReplicaRouter:
2     ...
3
4     def allow_relation(self, obj1, obj2, **hints):
5         """
6         All relations are allowed, since all DBs have the same data.
7         """
8         return True
9
10    def allow_migrate(self, db, app_label, model_name=None, **hints):
11        """
12        Only default can run migrations.
13        """
14        return bool(db == "default")
```



The final version

```
1 class ReadReplicaRouter:
2     def db_for_read(self, model, **hints):
3         """
4         Reads go to the Aurora-managed read replicas.
5         """
6         return "readonly"
7
8     def db_for_write(self, model, **hints):
9         """
10        Writes always go to the primary instance (writer).
11        """
12        return "default"
13
14    def allow_relation(self, obj1, obj2, **hints):
15        """
16        All relations are allowed, since all DBs have the same data.
17        """
18        return True
19
20    def allow_migrate(self, db, app_label, model_name=None, **hints):
21        """
22        Only default can run migrations.
23        """
24        return bool(db == "default")
25
```



Building the custom router

Last thing, we need to add the router to our **DATABASE_ROUTERS** setting in our Django settings file:

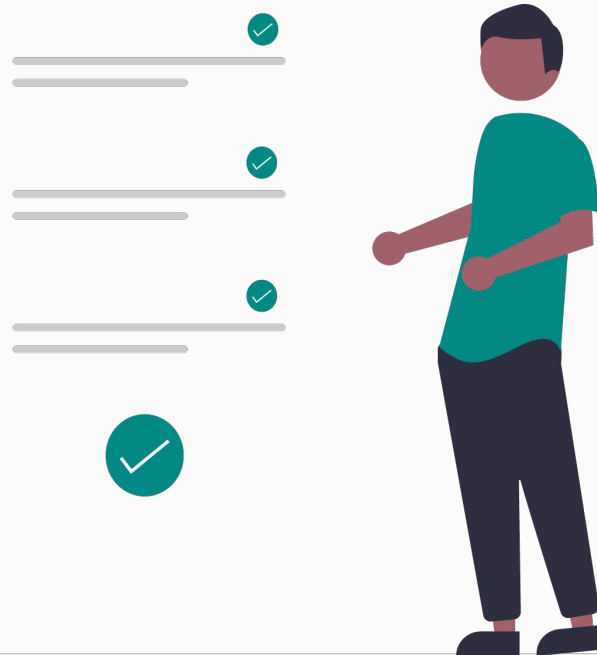


```
1 DATABASE_ROUTERS = ["core.database_routers.ReadReplicaRouter"]
```



Building the custom router

That's it! Your router is done and good to go.



Insights and gotchas

When to enforce the writer?

- **Read-after-write**

- POST → redirect → GET reads a replica inside the lag window
- Immediate re-read in the same request (`refresh_from_db`)
- Async task fired right after a write starts before lag catches up
- Session / auth: write then read on the next line

- **Locking**

- `select_for_update()` / `select_for_share()` - row locks are writes
- Advisory locks - lock table lives on the writer
- On a replica: read-only transaction error, not a silent bug



When to enforce the writer?



```
1  from django.db import transaction
2
3  # Inside a write transaction, reads must NOT go to the replica.
4  with transaction.atomic(using="default"):
5      account = (
6          Account.objects
7              .using("default")          # force this read onto the writer
8              .select_for_update()      # lock - only valid on the writer
9              .get(pk=account_id)
10     )
11     account.balance -= amount
12     account.save(using="default") # write stays on the same connection
13     # This read also stays consistent - same connection, same txn:
14     Account.objects.using("default").get(pk=account_id)
```



When to enforce the writer?

- **Decisions that gate a write → correctness**
 - Balance / limit checks before a debit
 - Idempotency & dedup pre-checks before insert
 - `get_or_create` / unique-constraint pre-reads
 - Exact counters, quotas, rate limits
- **Structural → never belongs on a replica**
 - Migrations and schema introspection (DDL)
 - Multi-step workflows → step N reads step N-1's write

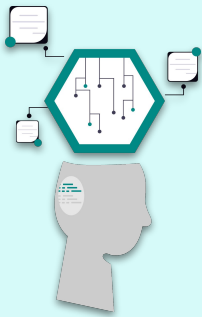


When to enforce the writer?

- What tolerates lag depends on your app and business
- **The real work:** analyzing each case and its trade-offs
- This analysis is the most important part of the whole design



When to enforce the writer?



AI Tip:

- Take advantage of custom skills and assisted code review to help identify writer-bound flows that might be overlooked during implementation.
- Use coding models to improve your ORM usage aiming for lag-impact reduction and writer/reader usage optimization.



Cost optimization

- Treat metrics separately for writer and readers → Scaling demands are different
- Take advantage of Auto-scaling
 - The management is free - you only pay for running instances + CloudWatch
 - Schedule scaling for predictable cycles (payroll days, business hours)
- Use savings plans / reserved instances for the baseline instances (pay attention to instance families)



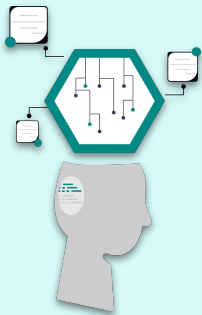
Cost optimization

	Provisioned	Serverless v2
Billing	Per instance-hour, fixed	Per ACU-hour (~\$0.12)
Best for	Steady, high utilization	Variable / spiky / idle
Discount lever	Reserved Instances (40-72%)	Database Savings Plans (~35%)
Floor	Always-on, always billed	Can scale to 0 (auto-pause)

- **Rule of thumb:** < 40% avg utilization (time) → Serverless; > 60% → Provisioned + RIs
- **Best of both is possible:** provisioned writer + Serverless v2 readers



Cost optimization



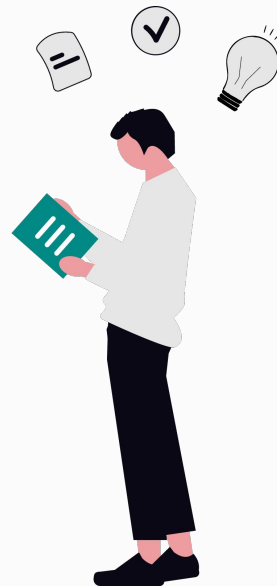
AI Tip:

- Use your usage metrics to enumerate possibilities and identify the best allocation and config of your resources.



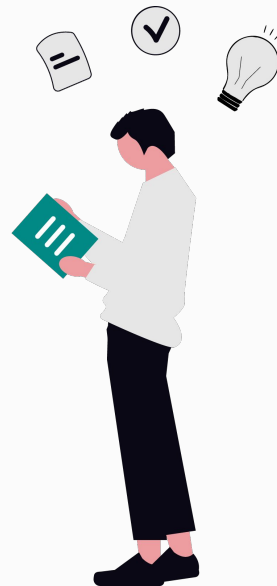
Lessons learned

- The router is simple, the real work is on consistency
- Auto-scaling readers saves, but sometimes may be tricky:
 - Connections may unexpectedly close when scaling in
 - Prefer using same instance family and size between writer and readers → More consistent metrics



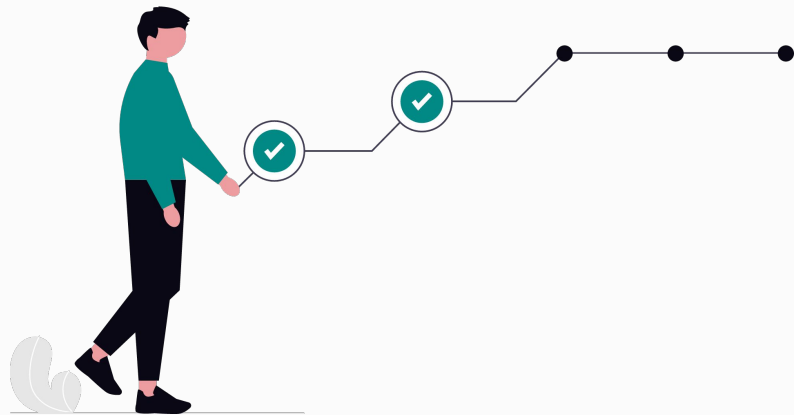
Lessons learned

- Aurora lag is usually milliseconds, but...
 - ...sometimes it isn't enough
 - ...sometimes it's more than that
- Avoid early optimization
 - Sometimes the right answer is one more year on a larger instance
 - Sometimes some indexes address +90% of the problem
 - Measure and understand your workload first



Where we are now

- DB Cluster → Provisioned, stable volumes
 - Writer
 - stable, good margin
 - covered by Reserved Instances
 - Readers
 - 2 fixed readers (covered by RI)
 - peaks trigger auto-scaling (on-demand)
- Django
 - **Continuous work** on adjusting writer-bound workloads and indexing
 - Current focus on reducing scale-in impacts (connection unexpectedly closed) and improving retry-on-error



Going further

What if I don't use AWS

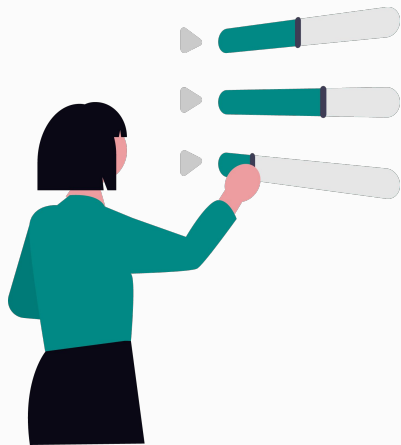
- GCP AlloyDB → Aurora's equivalent on Google Cloud, basically the same core architecture
- Azure Database for PostgreSQL Flexible Server
 - Virtual endpoints, but the read-only endpoint targets only one server at a time
 - Stable-DNS-on-failover benefit but not the read fan-out
- Implement the load balancing and domains on your own (You're really brave, congrats!)

WARNING: Based on a surface-level review of docs, not hands-on testing.

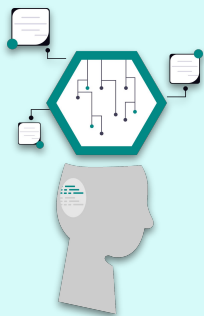


Adapting to your needs

- Multiple aliases → same database, different connection configs
 - Distinct roles (least-privilege), `statement_timeout`, `application_name`, isolation level
- Long-running reports on a separate connection (high timeout) vs OLTP (low timeout)
- Multi-tenancy via the router
 - Schema-per-tenant (`search_path` switch) or database-per-tenant
 - `db_for_read` / `db_for_write` pick the target from request context
- Sharding by tenant or key - same routing hook



Adapting to your needs



AI Tip:

- Use small models on your DB and backend metrics to dynamically shift traffic to the writer, reducing replication-lag side-effects during reader-usage peaks.



Thanks!

This presentation, my
contacts and more:



All illustrations are from [unDraw](#)